SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

*I **R. JAGAN-192321102** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: R. Jagan

Annexure A

Analytical Problem Solving

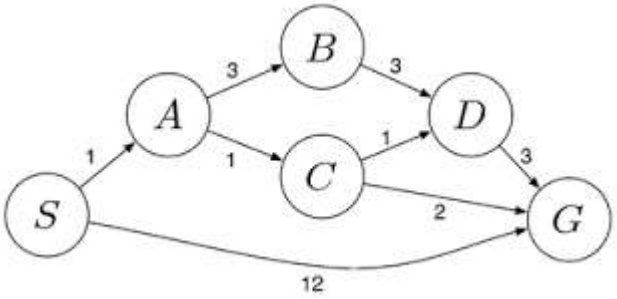
1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.
   The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

ANSWERS:

1.Water Jug Problem

1. Problem Understanding

We have a 4-gallon jug and a 3-gallon jug, both without markings, and a pump. The goal is to get exactly 2 gallons in the 4-gallon jug.

2. Problem Formulation

- **State:** (x, y), where x = water in the 4-gallon jug ($0 \leq x \leq 4$) and y = water in the 3-gallon jug ($0 \leq y \leq 3$).
- **Initial state:** (0, 0).
- **Goal state:** (2, n) for any n, i.e. exactly 2 gallons in the 4-gallon jug.
- **Actions:** fill a jug from the pump, empty a jug onto the ground, pour one jug into the other until the receiver is full or the source is empty.
- **Path cost:** 1 per action.

3. Production Rules

No.	Rule	Effect
1	(x, y) if $x < 4$	(4, y) – fill the 4-gallon jug
2	(x, y) if $y < 3$	(x, 3) – fill the 3-gallon jug
3	(x, y) if $x > 0$	(0, y) – empty the 4-gallon jug
4	(x, y) if $y > 0$	(x, 0) – empty the 3-gallon jug
5	(x, y) if $x + y \geq 4$ and $y > 0$	$(4, y - (4 - x))$ – pour 3-gal into 4-gal until full
6	(x, y) if $x + y \geq 3$ and $x > 0$	$(x - (3 - y), 3)$ – pour 4-gal into 3-gal until full
7	(x, y) if $x + y \leq 4$ and $y > 0$	(x + y, 0) – pour all of 3-gal into 4-gal
8	(x, y) if $x + y \leq 3$ and $x > 0$	(0, x + y) – pour all of 4-gal into 3-gal

4. Solution (Shortest Path, found by Breadth-First Search)

Step	State (x, y)	Action	Rule used
0	(0, 0)	Start	–
1	(4, 0)	Fill the 4-gallon jug from the pump	1
2	(1, 3)	Pour 4-gal jug into 3-gal jug until the 3-gal jug is full	6
3	(1, 0)	Empty the 3-gallon jug	4
4	(0, 1)	Pour the 1 gallon from 4-gal jug into 3-gal jug	8
5	(4, 1)	Fill the 4-gallon jug again	1
6	(2, 3)	Pour 4-gal into 3-gal until full (needs 2 gallons) – GOAL	6

Result: After 6 actions the state is (2, 3), so the 4-gallon jug holds exactly 2 gallons.

Alternative solution: (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0). It also takes 6 actions.

5. Interpretation

The state space is small (at most 5 × 4 = 20 states), so BFS finds an optimal 6-step solution. This shows how a real-world puzzle becomes a state-space search problem.

2.Mars Rover as an Intelligent Agent

The Mars Rover is a robotic agent that moves over Mars, collects soil and rock samples, analyses them and sends results to Earth.

(a) Percepts (2 Marks)

Percepts are the inputs the agent receives through its sensors:

- Camera images (stereo or panoramic) of terrain, rocks and obstacles
- Spectrometer and chemical-sensor readings of rock and soil composition
- Temperature, pressure, dust and wind sensors
- Wheel odometry, GPS-like localisation and gyroscope/accelerometer (tilt, slope, position)
- Battery and solar-panel power level, internal health and fault signals
- Commands and schedule from Earth control (delayed communication)

(b) Characterisation of the Environment (2 Marks)

Property	**Type**	**Reason**
Observability	Partially observable	Sensors see only nearby terrain; hidden rocks, craters and soil layers are unknown
Agents	Single agent	Only the rover operates (Earth control is a supervisor, not a competing agent)
Determinism	Stochastic	Wheel slip, sand, dust storms and sensor noise make outcomes uncertain
Episodic / Sequential	Sequential	Current moves affect later energy, position and mission goals
Static / Dynamic	Dynamic	Weather, light and temperature change while the rover acts
Discrete / Continuous	Continuous	Positions, speeds and sensor values are continuous
Known / Unknown	Partly unknown	Terrain is unmapped; the physics rules are known

(c) Actions (2 Marks)

- Move forward/backward, turn left/right, stop

- Climb, avoid or go around obstacles and craters
- Deploy robotic arm, drill, scoop or grind a sample
- Place a sample into the on-board analyser and run tests
- Capture and transmit images and data to Earth
- Orient solar panels, recharge, go into sleep/safe mode
- Request help or wait for commands when stuck

(d) Performance Measure (2 Marks)

- Number and scientific value of samples collected and analysed correctly
- Area explored and number of target sites reached
- Safety: no collisions, tip-overs or getting stuck
- Energy efficiency and battery life, mission duration
- Quality and timeliness of data sent to Earth
- Fewer faults and reduced human intervention

(e) Suitable Agent Architecture (2 Marks)

A hybrid, goal-based and utility-based learning agent with a layered architecture is most suitable.

- **Reactive layer (simple reflex):** immediate reactions such as emergency stop, avoid cliffs and tilt. Radio delay to Earth is up to about 20 minutes, so the rover cannot wait for commands.
- **Goal-based layer:** plans paths and sequences of tasks to reach a target rock and collect a sample.
- **Utility-based layer:** trades off competing aims such as science value against energy use and risk.
- **Learning element:** improves terrain classification and navigation from experience.

A pure simple reflex agent fails because the environment is only partially observable and needs memory and planning. A pure goal-based agent cannot handle risk and energy trade-offs. Hence the hybrid architecture is best.

3.8-Queens Problem Formulation and Search

1. Problem Understanding

Place 8 queens on an 8 × 8 chessboard so that no two queens share the same row, column or diagonal. The configuration in the question is not a solution because two queens lie on the same diagonal (first and last column).

2. Problem Formulation (Incremental Formulation)

Component	Definition
States	Any arrangement of 0 to 8 queens on the board. (Better: one queen in each of the leftmost n columns with no queen attacking another.)
Initial state	Empty board, no queens.
Actions	Place a queen in the leftmost empty column in a square that is not attacked by any queen already placed.
Transition model	Returns the board with the new queen added.
Goal test	8 queens are on the board and none is attacked.
Path cost	Not important (only the final configuration matters). Each placement can cost 1.

3. Search Space Size

- Naive formulation (place any queen on any square): $64 \times 63 \times \ldots \times 57 \approx 1.8 \times 10^{14}$ sequences.
- Improved formulation (one queen per column, not attacked): only 2,057 states. Search becomes practical.
- Total 92 distinct solutions (12 unique if rotations and reflections are treated as the same).

4. Exploring the Search Space

- **BFS / DFS (uninformed):** DFS with backtracking is best, as solutions lie at depth 8 and use little memory. BFS needs too much memory.
- **Backtracking:** place a queen column by column; if no safe row exists, go back and move the previous queen.
- **Local search (hill climbing, complete-state formulation):** start with 8 queens (one per column) and move a queen within its column to reduce the number of attacking pairs h.

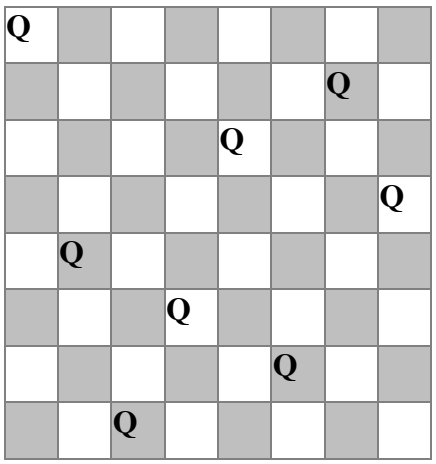
5. Backtracking Algorithm

```
function SOLVE(col):
   if col == 8: return True            # all queens placed
   for row in 0..7:
      if SAFE(row, col):               # no same row / diagonal
         place queen at (row, col)
         if SOLVE(col + 1): return True
         remove queen from (row, col) # backtrack
   return False

function SAFE(row, col):
   for each placed queen (r, c) with c < col:
      if r == row or |r - row| == |c - col|: return False
   return True
```

6. A Valid Solution

Rows of the queens in columns 1 to 8 are: (1, 5, 8, 6, 3, 7, 2, 4). No two share a row or a diagonal, so this is a valid goal state.



7. Interpretation

A good formulation (placing queens one per column and only in safe squares) reduces the search space enormously. This shows that problem formulation matters as much as the search algorithm.

4.OLA Cab Booking – Problem-Solving Agent

1. Type of Agent

A goal-based problem-solving agent is used, with a utility-based element for choosing the cab type. The agent has the goal of reaching the destination from the source. It formulates the problem, searches for a route and a suitable cab, and then executes the plan. The environment is fully observable (map and prices in the app), deterministic and discrete (approximately), so a search-based agent fits.

2. Problem Formulation

Component	OLA cab problem
Initial state	Customer at pickup location, no cab booked.
Goal state	Customer at the destination.
Actions	Select cab type (Mini, Micro, Sedan, Shared, Prime…), confirm booking, driver arrives, travel along a road to the next junction, end trip.
Transition model	Result of each action, e.g. Book(Sedan) → cab assigned; Travel(road) → new location.
Path cost	Fare, travel time, distance or waiting time (weighted by the customer's comfort preference).
Solution	A sequence of actions: choose a cab, ride a route from source to destination.

3. Utility Function for Cab Selection

Utility $U(cab) = w_1 \times comfort - w_2 \times fare - w_3 \times time$. The agent chooses the available cab type with the highest utility according to the customer's preferences.

4. Pseudocode

```
function OLA-AGENT(percept) returns an action
   persistent: seq (action sequence, initially empty),
               state, goal, problem

   state <- UPDATE-STATE(state, percept)   # location, cabs, prices
   if seq is empty then
```

```
    goal    <- FORMULATE-GOAL(state)    # reach destination D
    problem <- FORMULATE-PROBLEM(state, goal)
    cab     <- SELECT-CAB(state, preferences)
    seq     <- SEARCH(problem, cab)     # route search, e.g. A*
    if seq = failure then return NO-CAB-AVAILABLE
  action <- FIRST(seq)
  seq    <- REST(seq)
  return action

function SELECT-CAB(state, pref)
  best <- null ; bestU <- -infinity
  for each cab in {Mini, Micro, Sedan, Shared, Prime, ...}:
    if AVAILABLE(cab, state.location):
      U <- w1*COMFORT(cab) - w2*FARE(cab) - w3*ETA(cab)
      if U > bestU then best <- cab ; bestU <- U
  return best

function SEARCH(problem, cab)              # uniform-cost / A*
  node <- NODE(problem.INITIAL) ; frontier <- {node}; explored <- {}
  while frontier is not empty do
    node <- POP lowest f(n) from frontier
    if GOAL-TEST(node.state) then return SOLUTION(node)
    add node.state to explored
    for each action in ACTIONS(node.state) do
      child <- CHILD-NODE(problem, node, action)
      if child.state not in explored/frontier then add child
  return failure
```

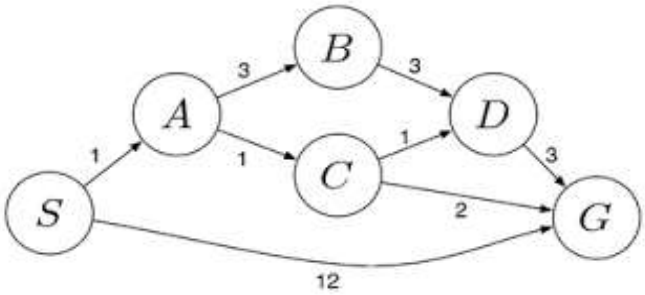
5. Working

- Percept: pickup, drop location, available cabs, prices, traffic.
- Goal formulation: reach the destination at acceptable cost and comfort.
- Search: finds the best sequence of road segments and cab choice.
- Execution: book the cab, track the ride and end the trip. If conditions change (traffic, cancellation), the agent re-plans.

5.Uniform Cost Search (UCS) – Warehouse Network

1. Problem Understanding

Find the least-cost path from the start warehouse S to the destination G in the weighted directed graph shown below. UCS always expands the node with the lowest path cost g(n) from the priority queue (frontier).



Edges: S→A = 1, S→G = 12, A→B = 3, A→C = 1, B→D = 3, C→D = 1, C→G = 2, D→G = 3.

2. Method Selected

Uniform Cost Search is complete and optimal when all step costs are positive. It is the right method here because the edges have different costs and we need the minimum total cost, not the fewest edges.

3. Solution Process (Step by Step)

Step	Node expanded (cost g)	Frontier after expansion (node : g)	Remarks
1	S (0)	A:1, G:12	Start with S
2	A (1)	C:2, B:4, G:12	S→A→B = 4, S→A→C = 2
3	C (2)	D:3, B:4, G:4	G now 4 via C (2+2), replaces 12; D = 2+1 = 3
4	D (3)	B:4, G:4	D→G = 3+3 = 6, not better than 4
5	B (4)	G:4	B→D = 7, worse than D:3
6	G (4)	–	Goal popped: path found, STOP

Order of expansion: S → A → C → D → B → G (ties at cost 4 broken in FIFO order).

4. Final Answer

Optimal path	**S → A → C → G**
Total cost	1 + 1 + 2 = 4 units

5. Verification of Other Paths

Path	**Cost**	**Result**
S → A → C → G	1+1+2 = 4	Optimal
S → A → C → D → G	1+1+1+3 = 6	Costlier
S → A → B → D → G	1+3+3+3 = 10	Costlier
S → G (direct)	12	Costliest

6. Interpretation

The shortest-looking route S → G (one edge) costs 12, the most expensive one. The cheapest route uses three edges for a total cost of 4 units (fuel + time). UCS guarantees this is optimal because a goal is accepted only when it is popped from the queue with the lowest cost, not when it is first generated. Time complexity is O(b^(1+⌊C*/ε⌋)) with C* = 4 here.